DRIFT · Reto del segundo corte

Arquitectura de Software AS_202620 · Escenario E6: degradación de una fuente bajo carga · 11 de octubre de 2026
Repositorio: github.com/ISCOUTB/AS_202620_Drift · Despliegue: https://drift.iscoutb.dev

1. Punto débil e hipótesis

En el despliegue del laboratorio, 4 de 11 búsquedas tardaron 5,4 s y llegaron sin los precios de CheapShark y sin aviso. Una respuesta a la que le falta una tienda se ve igual que una completa y puede señalar un "mejor precio" equivocado. El código lo explicaba: el combinador de fuentes esperaba a la fuente secundaria sin plazo y descartaba sus errores en silencio, y los adaptadores de Steam y CheapShark retenían un lock durante la llamada HTTP, de modo que las búsquedas con caché fría se atendían una por una.

Hipótesis. Con 50 usuarios concurrentes haciendo búsquedas distintas, caché fría y CheapShark tardando más de 5 s en cerca del 35 % de las consultas, si la búsqueda libera el lock durante la E/S, espera a cada fuente secundaria como máximo 3 s y registra la fuente faltante, entonces: **p95 ≤ 5 s, fallidas < 1 % y 0 respuestas degradadas sin aviso**.

2. Decisión

Registrada en **ADR-0008**, que sustituye a ADR-0004. La sustitución se declara en el propio ADR-0008; ADR-0004 no se edita. Dos decisiones:

- **Mantener catálogo en memoria y ejecución síncrona.** No existe adaptador ni servicio de MySQL; DRIFT no guarda datos propios y el catálogo se reconstruye desde las fuentes. MySQL añadiría un contenedor dentro de la cuota de 512 MB sin un requisito que lo exija hoy.
- **Degradación controlada, solo en infraestructura** (dominio, puertos y casos de uso no cambian): `SingleFlightCache` (lock fuera de la E/S y una carga en vuelo por clave), plazo de 3 s por fuente secundaria (`DRIFT_SOURCE_TIMEOUT_SECONDS`) y aviso por fuente en `unavailable_sources`.

Alternativas descartadas: dejar el estado actual (mantiene el problema); circuito de corte con biblioteca externa (más dependencias para lo que un plazo resuelve); MySQL con actualización asíncrona (la de ADR-0004, sin requisito que la justifique). **Costo:** sin infraestructura nueva; se acepta que las búsquedas afectadas esperen los 3 s completos y que un hilo abandonado viva hasta su timeout HTTP.

3. Ejecución

Carga	k6 v2.2.0 · 50 usuarios virtuales, 1 búsqueda distinta cada uno · 1 worker de `uvicorn`
Fuentes	Simuladas (`scripts/stub_upstream.py`) con latencias fijas: Steam 0,15 s, CheapShark 0,25 s, PlayStation 0,25 s
Degradación	CheapShark tarda 6 s en 18 de 50 consultas (36 %); el mismo conjunto en todas las corridas
Versiones	Antes: commit `ab591de`. Después: el cambio de ADR-0008. Misma envoltura (`scripts/e6_backend.py`)
Repetición	3 corridas por versión, intercaladas, cada una con procesos nuevos (caché fría): `python scripts/e6_corrida.py <backend> <nombre>`

4. Resultado

Versión	Corrida	p50 (s)	p95 (s)	Máx (s)	Fallidas	Degradadas	Sin aviso	Cumple
Antes	1	52,09	93,54	98,94	0	18	18	No
Antes	2	59,28	98,34	99,00	0	18	18	No
Antes	3	49,74	86,69	98,97	0	18	18	No
Después	1	0,70	3,62	3,64	0	18	0	Sí
Después	2	0,82	3,51	3,65	0	18	0	Sí
Después	3	0,68	3,61	3,64	0	18	0	Sí

Mediana de 3 corridas	Antes	Después	Umbral
p95 de la búsqueda	93,54 s	3,61 s	≤ 5 s
p50 de la búsqueda	52,09 s	0,70 s	—
Degradadas sin aviso por corrida	18	0	0
Solicitudes fallidas (3 corridas)	0	0	< 1 %

> **Contraste.** Antes del cambio el escenario no se cumple: p95 mediano de 93,54 s y 18 respuestas degradadas sin aviso por corrida. Después se cumple en las tres corridas: p95 mediano de 3,61 s y ninguna respuesta degradada sin aviso. El p95 posterior ronda los 3,6 s porque las búsquedas afectadas esperan el plazo completo de 3 s.

5. Factores de confusión y límites de validez

- **Controlados:** latencia de los servicios reales (upstream simulado), conjunto de consultas lentas (determinista), caché caliente (procesos nuevos y consultas distintas) y orden (corridas intercaladas).
- **Costo del cliente HTTP:** en el equipo de medición `httpx.get` gasta ~0,5 s de CPU por llamada creando su contexto TLS. Se excluyó con una conexión simple, igual para ambas versiones, y quedó como deuda en arc42 §11.7.
- **Límites:** fuentes simuladas; un solo equipo local, no el servidor del laboratorio (0,5 CPU); el 36 % es un supuesto apoyado en una sola observación de 4 de 11 búsquedas; 50 solicitudes por corrida; no se probó degradación sostenida.

6. Operación, trazabilidad y estado

- **Observabilidad:** `/metrics` expone `drift_source_unavailable_total` (búsquedas degradadas y conteo por fuente); log JSON `source_unavailable` con fuente, motivo y espera.
- **Pruebas:** 27 del backend en verde; 7 nuevas en `test_source_degradation.py` y 2 de arquitectura que fallan si el núcleo importa infraestructura.
- **Documentación:** escenario E6, matriz de trazabilidad, ADR-0008 (sustituye a ADR-0004, que se conserva sin cambios), arc42 §11.7 y evidencia en `docs/evidencias/e6-degradacion-bajo-carga.md` con los archivos de k6.
- **Despliegue:** https://drift.iscoutb.dev verificado el 2026-10-12T04:35Z (portada, salud, búsqueda de "portal 2" en 0,89 s y compatibilidad, todos 200). En ese momento ejecutaba la versión anterior al cambio; el cambio se despliega al integrarlo en `master`.